

MCA
First Semester
“ Discrete Structure ”
Tree

Introduction to Tree

- A tree is a collection of nodes connected by directed(or undirected) edges.
- The relation between tree and graph is; “*a tree is a graph having no cycle(loop)*”.
- Tree represent the hierarchical relationship between various elements.
- In tree:
 - There is a special data item at the top of hierarchy called the **Root** of tree.
 - The remaining data items are partitioned into number of mutually exclusive subsets, which is called the subtree.
 - The tree always grows in length towards bottom in data structure,unlike natural trees which grow upwards.

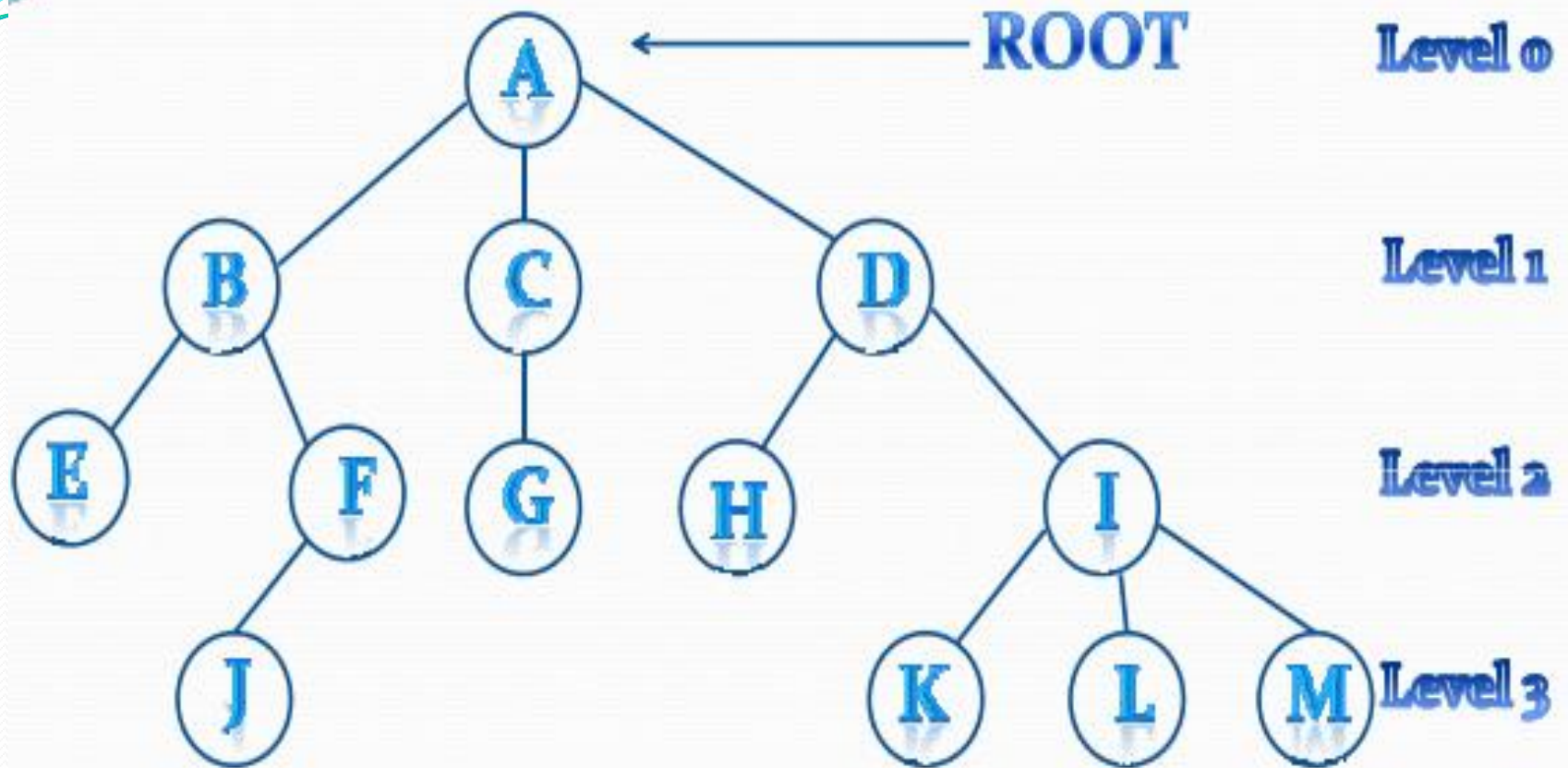


Fig: A Tree

Tree Terminologies

- **Node**: Each data item in a tree is called a node. It is the basic structure in a tree. It specifies the data and links (branches) to other data items. There are 13 nodes in the above tree.
- **Root**: It is the first data item (or node) in the hierarchical arrangement of data items. In the above tree, A is the root item.
- **Degree of a node**: It is the number of subtrees of a node in a given tree. For example, in the above tree,
 - The degree of node A is 3
 - The degree of node B is 2
 - The degree of node C is 1
 - The degree of node H is 0
 - The degree of node I is 3

Terminologies...

- **Degree of a tree:** It is the maximum degree of nodes in a given tree. In the above tree, the degree of node A is 3 and another node I also have degree 3. In the whole tree, this value is the maximum. So, the degree of the above tree is 3.
- **Terminal node(s):** A node with degree zero is called a **terminal node** or **leaf**. In the above tree, there are 7 terminal nodes: E, J, G, H, K, L and M.
- **Non-terminal node(s):** Any node (except the root node) whose degree is not zero is called **non-terminal node**. **Non-terminal nodes** are the intermediate nodes in traversing the given tree from its root node to the terminal nodes (leaves). In the above tree, there are 5 **non-terminal nodes**: B, C, D, F and I.

Terminologies...

- **Siblings**: The children nodes of a given parent node are called **siblings**. They are also called **brothers**. In the above tree,
 - E and F are siblings of parent node B
 - K, L and M are siblings of parent node I
- **Level**: The entire tree structure is leveled in such a way that the root node is always at level 0. Then its immediate children are at level 1 and their immediate children are at level 2 and so on up to the terminal nodes. In general, if a node is at level n , then its children are at level $n+1$. In the above tree, there are four levels from level 0 to level 3.
- **Edge**: It is a connecting line of two nodes. That is, the line drawn from one node to another node is called an **edge**.

Terminologies...

- **Path:** It is a sequence of consecutive edges from source to destination node. In the above tree, the **path** between A and J is given by the node pairs: (A, B), (B, F) and (F, J).
- **Depth:** It is the maximum level of any leaf in a given tree. This equals the length of the longest path from root to the terminal nodes (leaves). The term **height** is also used to denote the depth. The **depth** of the above tree is 3.
- **Forest:** It is a set of disjoint trees. In a given tree, if we remove its root, then it becomes a forest. In the above tree, there is forest with three trees if we remove the root node.

Binary Tree

- A binary tree is the most commonly used non-linear data structure.
- A binary tree is a finite set of data items which is either empty or is partitioned into three disjoint subsets. The first subset consists of a single data item called the **root** of the binary tree. The other two subsets are themselves binary trees called the **left subtree** and **right subtree** of the original binary tree.
- In a binary tree, the maximum degree of any node is **at most two**. This means that there may be a zero degree node (usually an empty tree or leaf) or a one degree node or a two degree node.

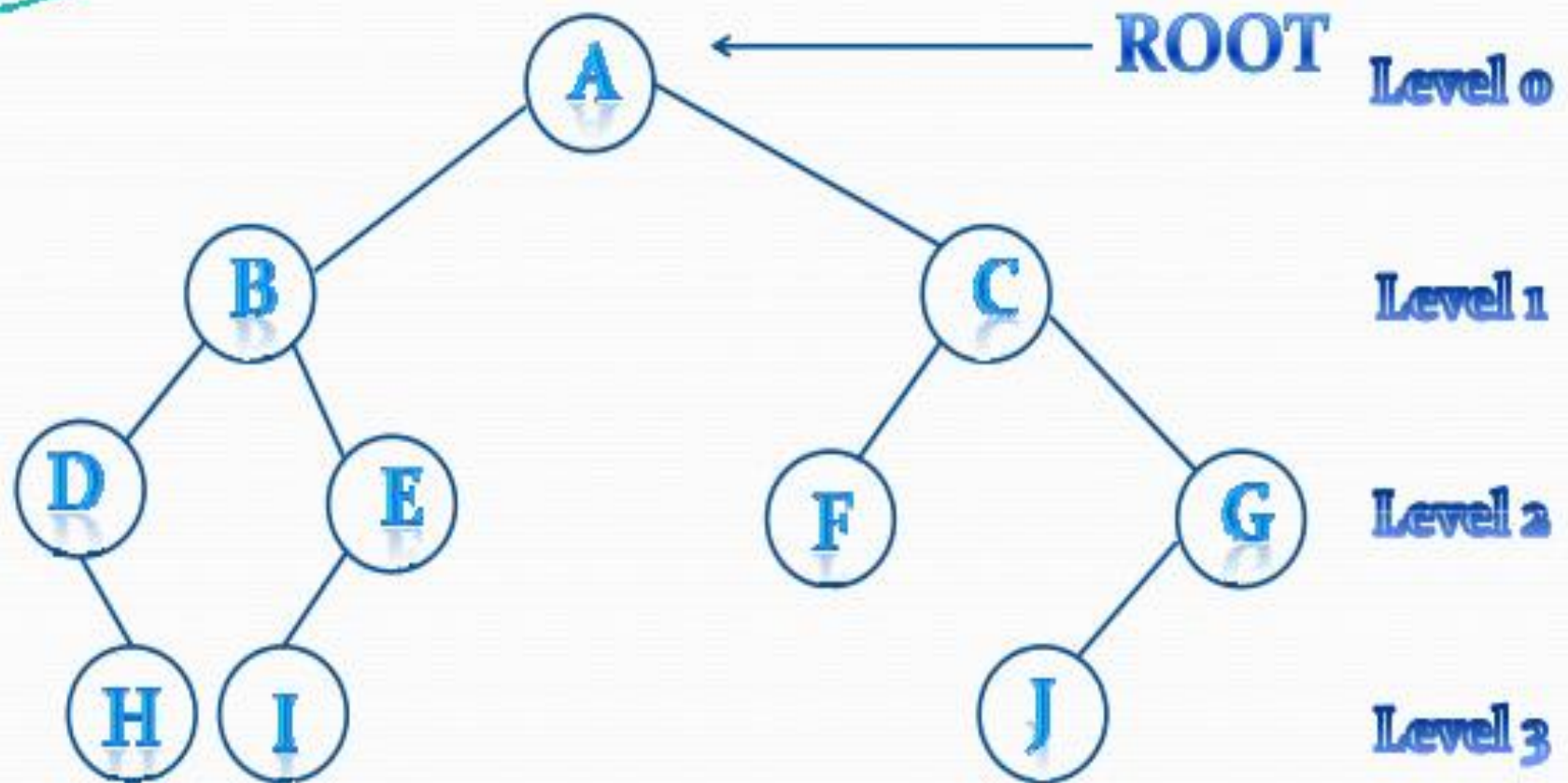


Fig: A Binary Tree

- In the above binary tree, **A** is the **root** of the **binary tree**. The **left subtree** consists of the tree with root **B** and the **right subtree** consists of the tree with root **C**. Further **B** has its **left subtree** with root **D** and **right subtree** with root **E**. Similarly, **C** has its **left subtree** with root **F** and its **right subtree** with root **G**. In the next level, **D** has an empty **left subtree** and its **right subtree** with root **H**. Similarly, **E** has a **left subtree** with root **I** and has no **right subtree**. **F** has neither its **left subtree** nor its **right subtree**. Finally, **G** has its **left subtree** with root **J** and it has no **right subtree**.

- **Note:**
- If **A** is the root of a binary tree and **B** is the root of its left or right subtree, then **A** is called **father** of **B** and **B** is called **left** or **right son** of **A**.
- A node that has no sons (such as **H**, **I**, **F** or **J**) is called a **leaf**.
- Node n_1 is an **ancestor** of node n_2 (and n_2 is a **descendant** of n_1) if n_1 is either the father of n_2 or the father of some ancestor of n_2 . For example, in the above binary tree **A** is an ancestor of **I**, and **J** is a descendant of **C**, but **E** is neither an ancestor nor a descendant of **C**.
- A node n_2 is a **left descendant** of node n_1 if n_2 is either the left son of n_1 or a descendant of the left son of n_1 . Similarly **right descendant** can be defined.

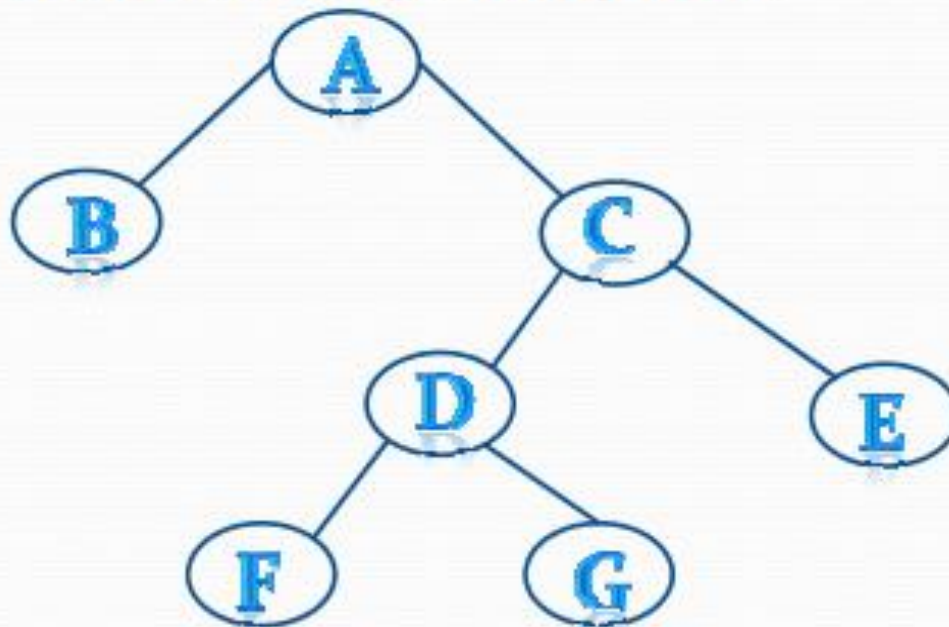


Types of Binary Trees:

- a) Strictly Binary Tree
- b) Complete(Full) Binary Tree

Strictly Binary Tree

- If every non-terminal node in a binary tree consists of non-empty left and right subtrees, then such a tree is called **strictly binary tree**.



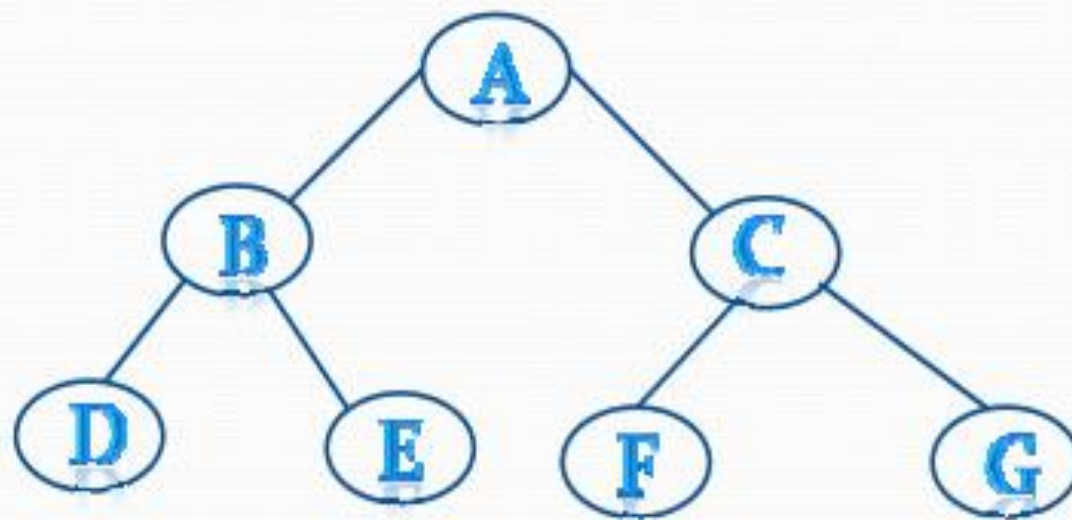
In this binary tree, all the non-terminal nodes such as C and D have non-empty left and right subtrees.

Note: A strictly binary tree with n leaves always contains $(2n-1)$ nodes.

Fig: A Strictly Binary Tree

Complete Binary Tree

- A **complete binary tree** of depth d is the **strictly binary tree** all of whose leaves are at level d .
- In a **complete binary tree**, there is exactly one node at level 0, two nodes at level 1, four nodes at level 2 and so on.



Note: A complete binary tree of depth d contains a total of $(2^{d+1}-1)$ nodes with 2^d leaves at level d and 2^d-1 non-leaf nodes.

Fig: A Complete Binary Tree

Binary Search Tree (BST)

- A **binary search tree** is a *binary tree* that is either empty or in which each node possesses a key that satisfies the following three conditions:
 - i. For every node X in the tree, the values of all the keys in its left subtree are smaller than the key value in X.
 - ii. Keys in its right subtree are greater than the key value in X.
 - iii. The left and right subtrees of the root are again binary search trees.

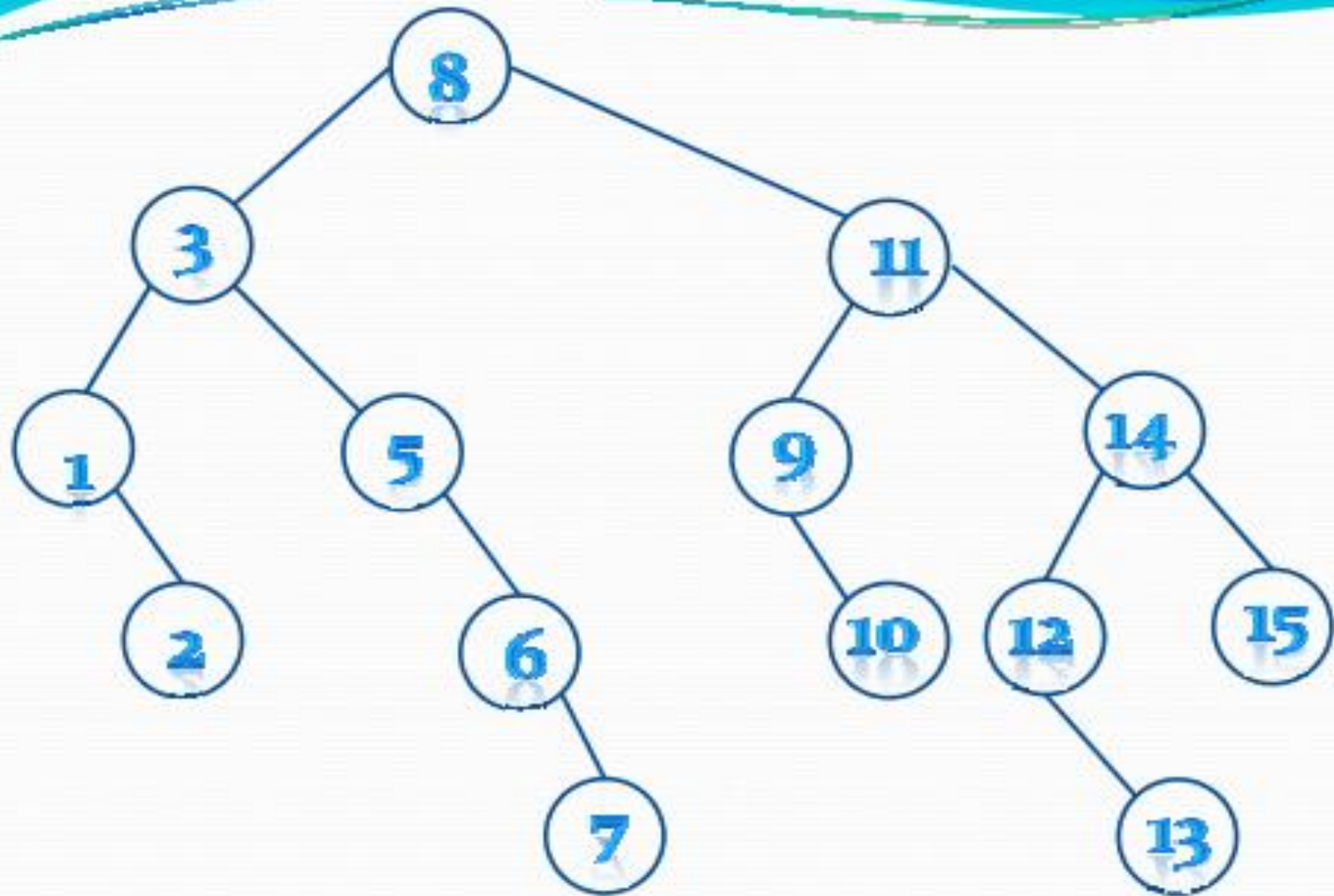


Fig: A Binary Search Tree

Classwork

Suppose the following eight numbers are inserted in order into an empty *binary search tree* T:

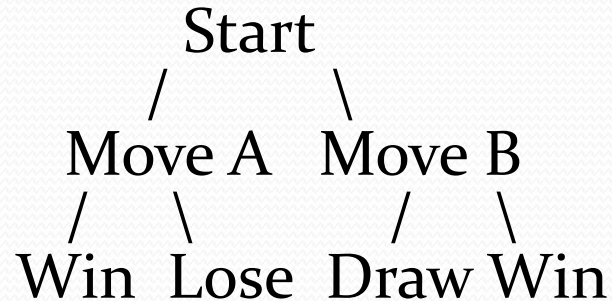
50, 33, 44, 77, 35, 60, 40, 100

Draw the tree T.

Game Tree

A **Game Tree** is a tree structure that represents all possible moves in a game from a given position.

Example (Tic-Tac-Toe)



Here, each node represents a game state, and each branch represents a possible move.

Applications

- Chess
- Tic-Tac-Toe
- Artificial Intelligence game playing

Huffman Tree

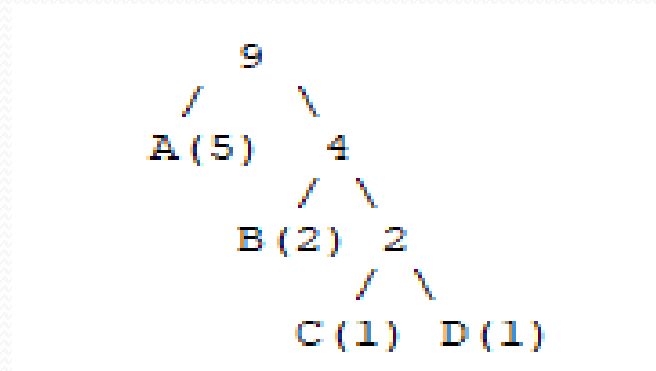
A **Huffman Tree** is a binary tree used for data compression by assigning shorter codes to frequently occurring characters.

Example

Characters and frequencies:

Character	Frequency
A	5
B	2
C	1
D	1

Huffman Tree:



Huffman Tree

Possible Codes:

Character	Code
A	0
B	10
C	110
D	111

Applications

- ZIP files
- File compression
- Data transmission

Expression Tree

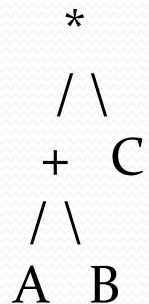
An **Expression Tree** is a binary tree used to represent arithmetic expressions. Internal nodes contain operators, and leaf nodes contain operands.

Example

Expression:

$(A + B) * C$

Expression Tree:



Applications

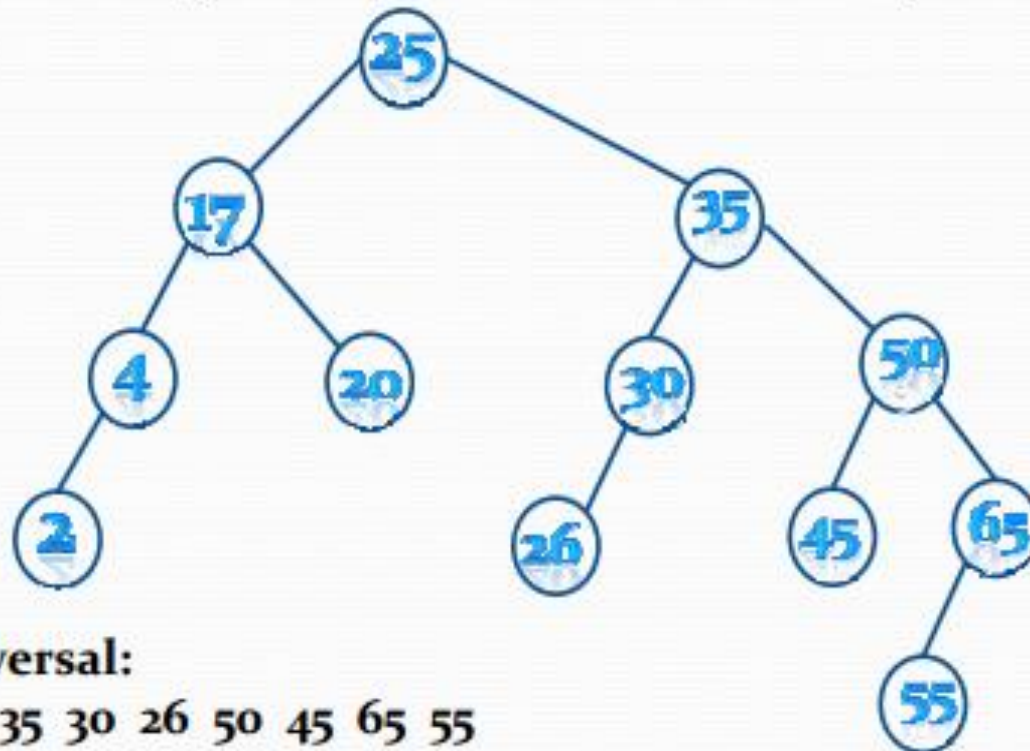
- Compilers
- Expression evaluation
- Calculator software

Tree Traversal

- “*Traversing a binary tree is a way in which each node in the tree is visited exactly once in a systematic manner.*”
- We know that the order of traversal of nodes in a linear list is from first to last, however there is no such “natural” linear order for the nodes of a tree.
- We have the following three orderings or methods for traversing a **non-empty** binary tree:
 - *Preorder (Depth-first order) Traversal*
 - *Inorder (Symmetric order) Traversal*
 - *Postorder Traversal*
- All these traversal techniques are defined ***recursively***.

Preorder Traversal

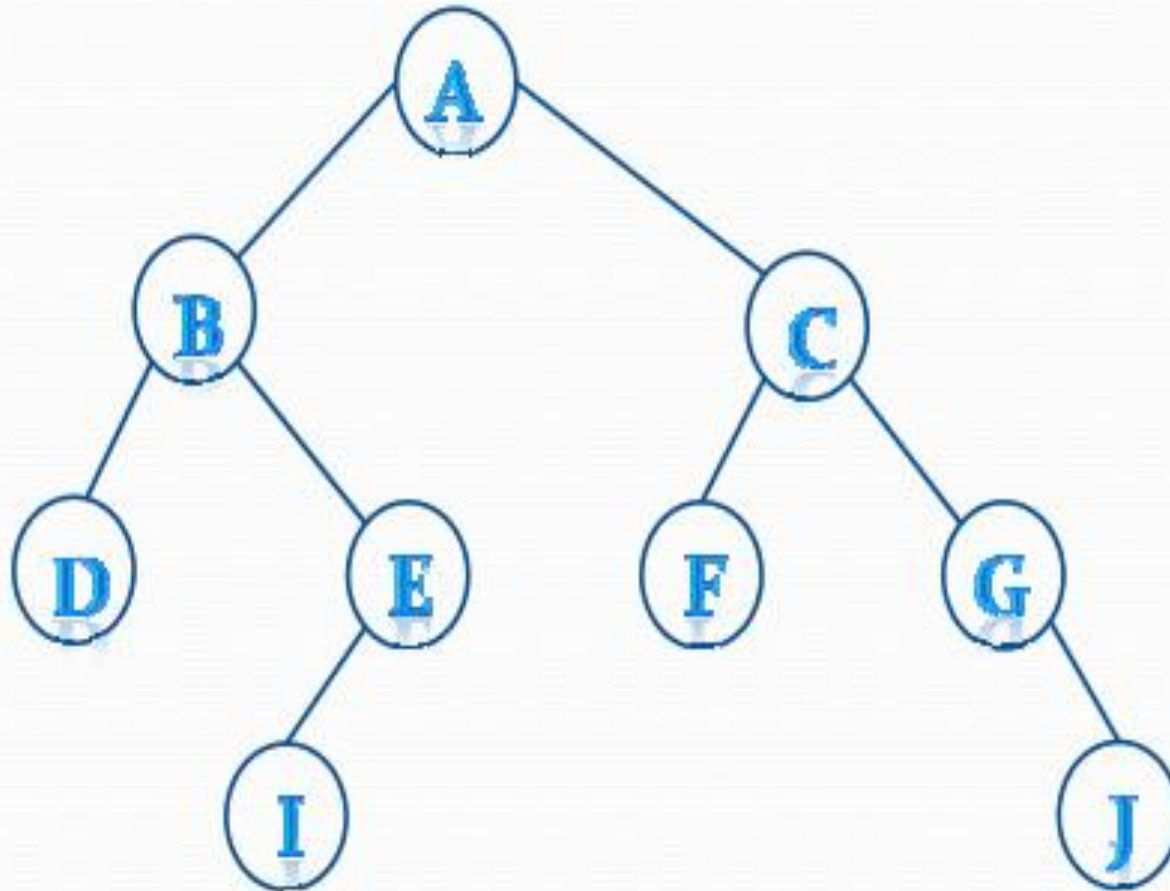
- In this technique, first of all we process the root **R** of the binary tree **T**. Then, we traverse the left subtree **T₁** of **R** in preorder (which means that we traverse root of subtree **T₁** first and then its left subtree). After visiting left subtree of **R**, then we take over right subtree **T₂** of **R** and process all the nodes in preorder.



Preorder Traversal:

25 17 4 2 20 35 30 26 50 45 65 55

Traverse the given binary tree in preorder



Answer: A B D E I C F G J

Algorithm for preorder traversal

1. If root=NULL, return.
2. Visit the root.
3. Traverse the left subtree in preorder.
4. Traverse the right subtree in preorder.

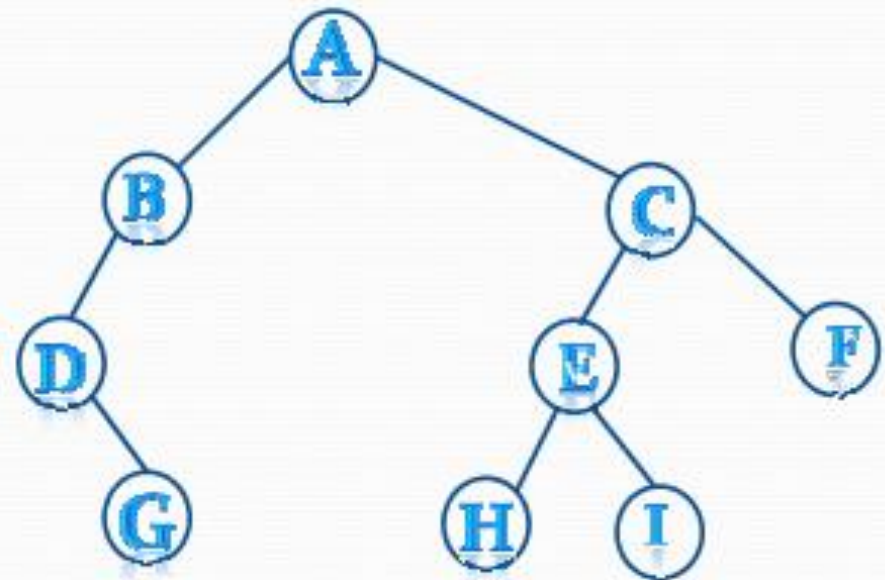
C function for preorder traversal

- Function Call: *preorder(&rootnode);*
- Function Definition

```
void preorder(struct binarytree **root)  
{  
    if(*root==NULL)  
        return;  
    printf("%d\t", (*root)->info);  
    preorder(&(*root)->leftlink);  
    preorder(&(*root)->rightlink);  
}
```


Postorder Traversal

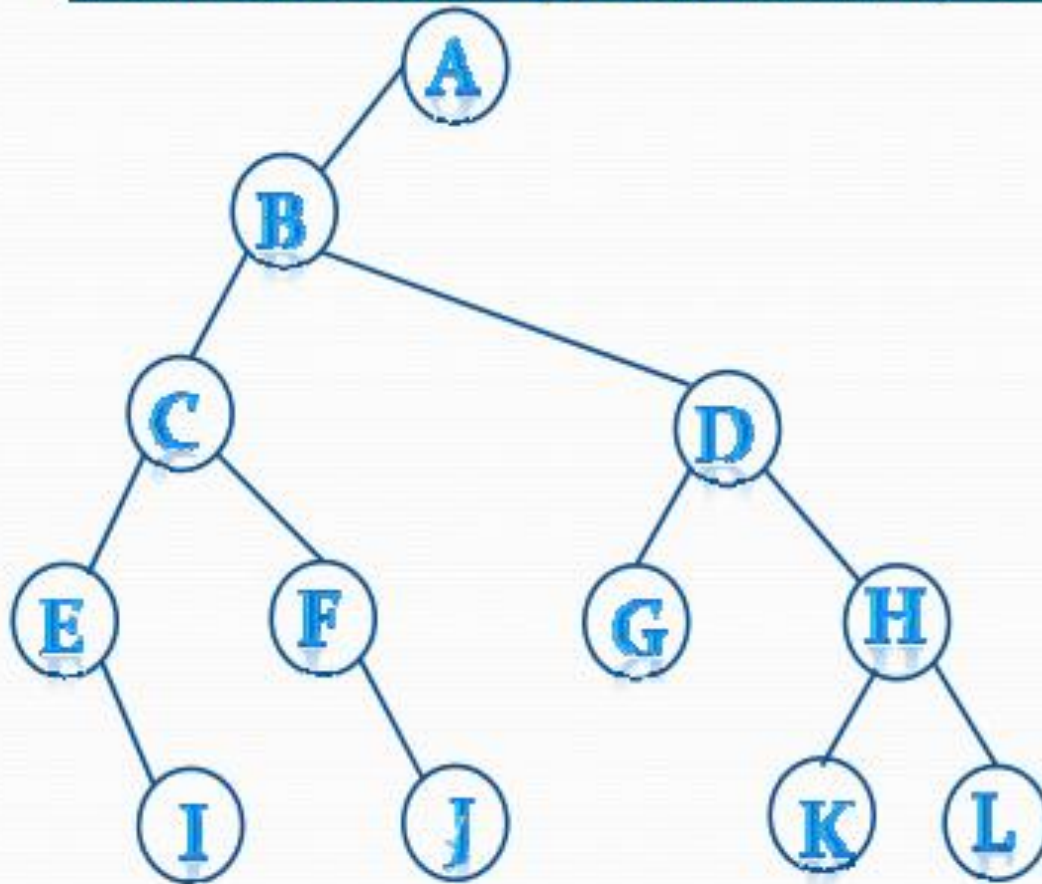
- In this technique, first of all we process the left subtree T_1 of R in postorder, then right subtree T_2 of R in postorder and at the last, the root R .*



Postorder Traversal:
G D B H I E F C A

Preorder Traversal:
A B D G C E H I F

Traverse the given binary tree in postorder



Answer: I E J F C G K L H D B A

Preorder: A B C E I F J D G H K L

Algorithm for postorder traversal

1. If root=NULL, return.
2. Traverse the left subtree in postorder.
3. Traverse the right subtree in postorder.
4. Visit the root.

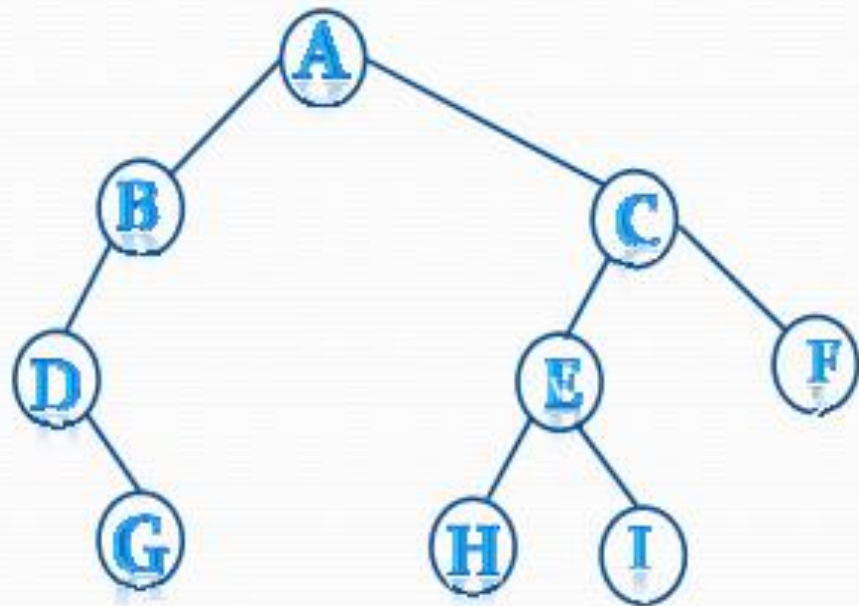
C function for postorder traversal

- Function Call: **postorder(&rootnode);**
- Function Definition

```
void postorder(struct binarytree **root)  
{  
    if(*root==NULL)  
        return;  
    postorder(&(*root)->leftlink);  
    postorder(&(*root)->rightlink);  
    printf("%d\t", (*root)->info);  
}
```

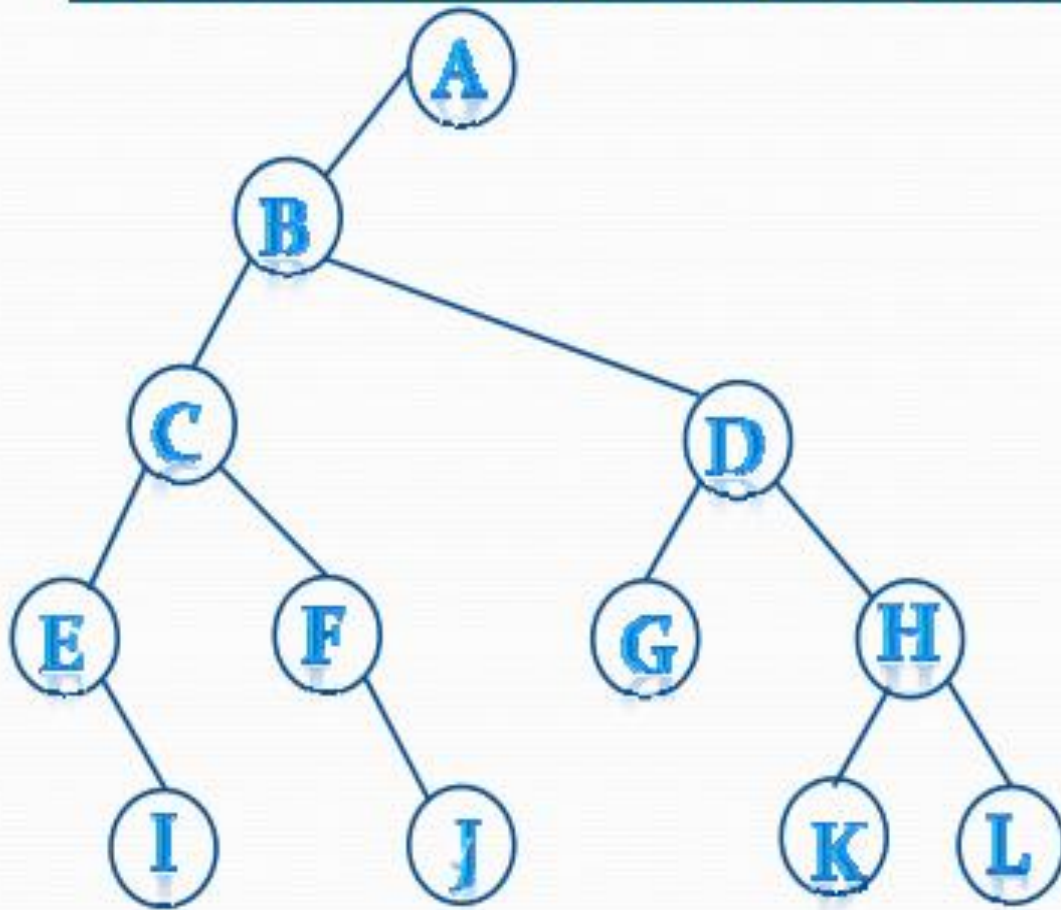

Inorder Traversal

- In this traversal technique, first of all we process the left subtree T_1 of the root R in inorder, then we process the root and at last the right subtree T_2 of R .*



Inorder Traversal:
D G B A H E I C F

Perform inorder traversal on the given tree



Answer: E I C F J B G D K H L A

Algorithm for inorder traversal

1. If root=NULL, return.
2. Traverse the left subtree in inorder.
3. Visit the root.
4. Traverse the right subtree in inorder.

C function for inorder traversal

- Function Call: ***inorder(&rootnode);***
- Function Definition

```
void inorder(struct binarytree **root)  
{  
if(*root==NULL)  
    return;  
inorder(&(*root)->leftlink);  
printf("%d\t", (*root)->info);  
inorder(&(*root)->rightlink);  
}
```


Note:

- By traversing a **binary search tree (BST)** in inorder, the nodes are always traversed (and therefore printed) in ascending order.
- If we want to traverse and print data in descending order in a **binary search tree**, then follow following steps:
 1. *Traverse right subtree in inorder.*
 2. *Visit the root.*
 3. *Traverse the left subtree in inorder.*

Homework

Suppose the following list of letters is inserted in order into an empty *binary search tree* T:

S, T, P, Q, M, N, O, R, K, V, A, B

- a) Draw the final tree T.
- b) Find the inorder traversal of tree T.

Answer:

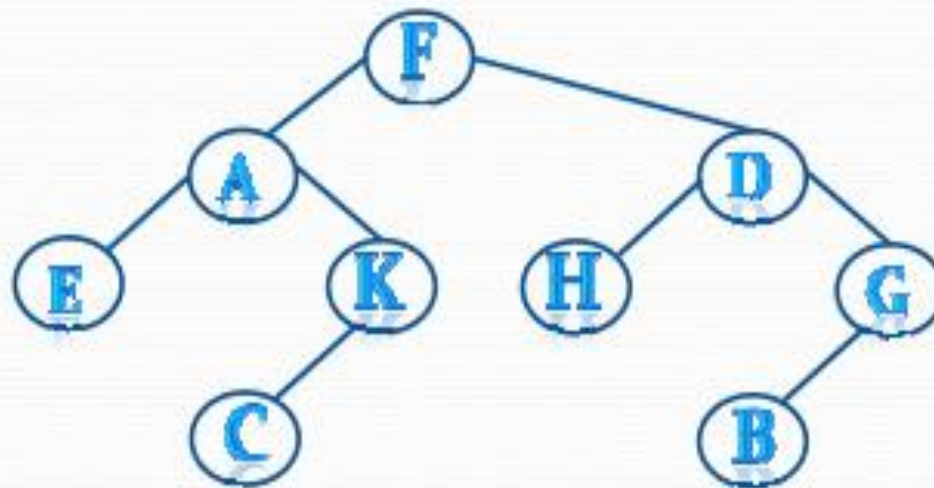


Fig: Final Binary Tree

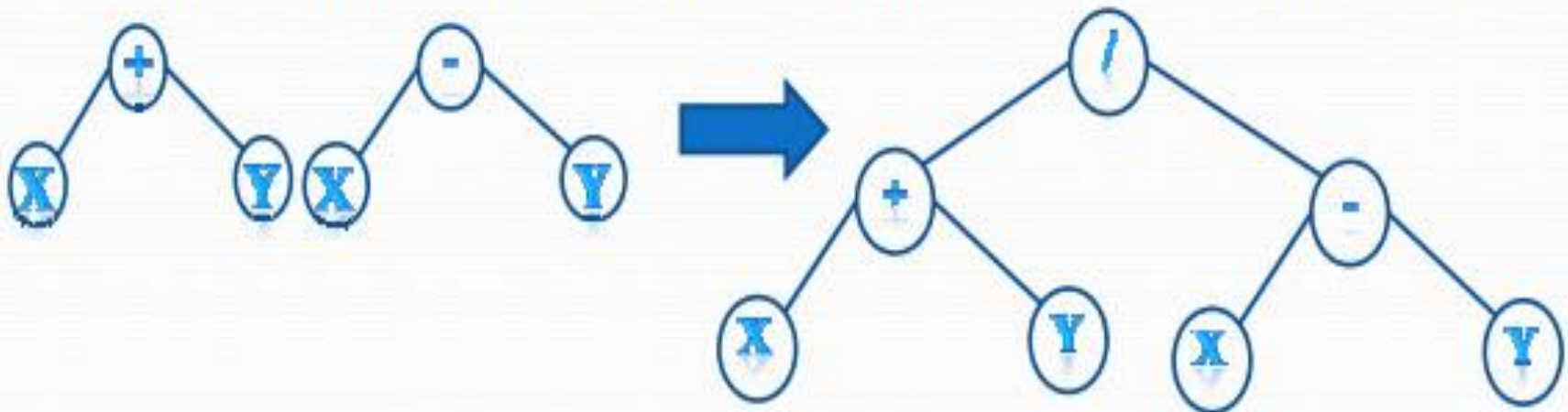
Application of Binary Tree

- A **binary tree** (or more appropriately **strictly binary tree**) can be used to represent an expression containing operands and binary operators.
- A **binary tree** used to represent a binary expression is called a **binary expression tree (BET)**.
- The root of **BET** contains the **operator** and its two children contains **operands**.
- For example, $X + Y$ can be represented as:



Application of Binary Tree...

- If we have an expression with more arithmetic operators like $(X+Y)/(X-Y)$, then we have to construct two subtrees and then these two subtrees are added to the root node as:



- When a **binary expression tree** is traversed in preorder (root-left-right), we get prefix form of the expression and when traversed in postorder (left-right-root), we get postfix form of the expression. Thus postfix form of above arithmetic expression is: $XY+XY-/$

Classwork

- Represent the following arithmetic expressions by their corresponding binary trees and find out their postfix equivalent:
 - a) $A+B*C$
 - b) $(A+B)*C$
 - c) $A+(B-C)*D$(E*F)$
 - d) $(A+B*C)$((A+B)*C)$

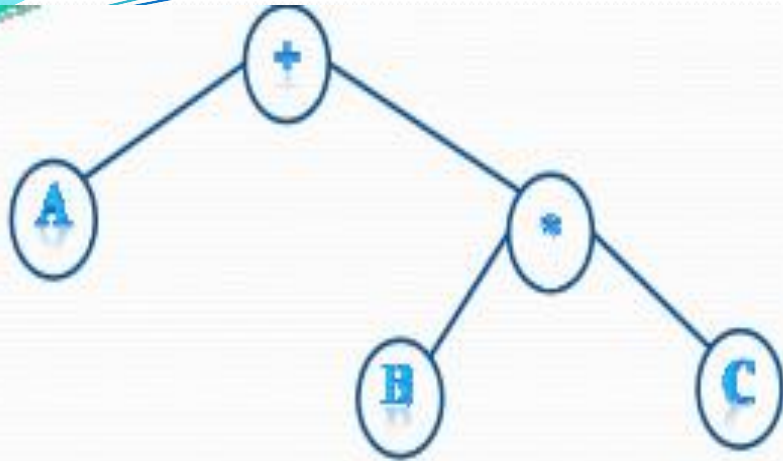


Fig (a): $A + B * C$
Postfix: ABC^*+

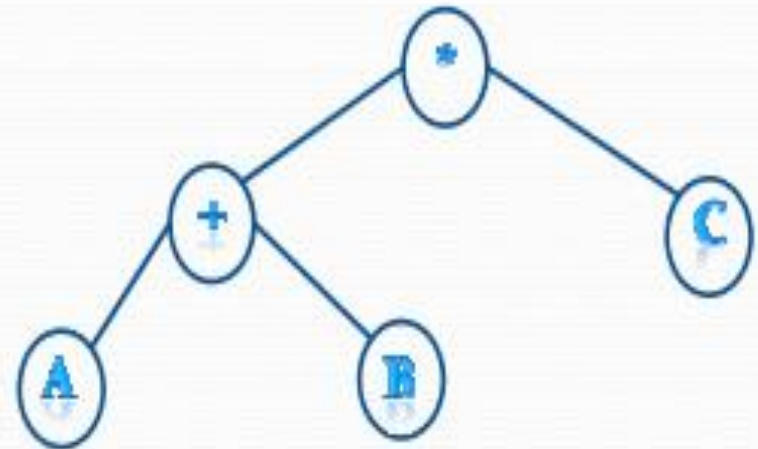


Fig (b): $(A + B) * C$
Postfix: $AB+C^*$

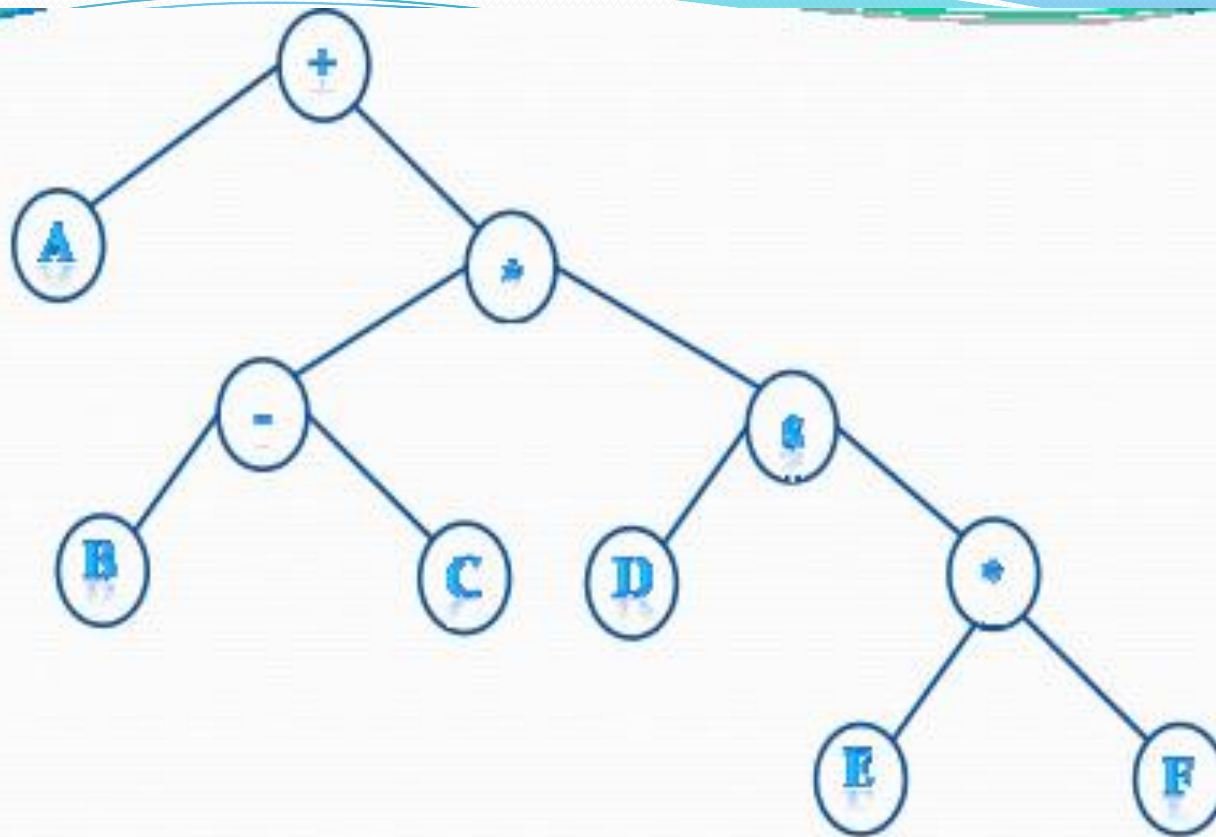


Fig (c): $A + (B - C) * D \$ (E * F)$

Postfix: $ABC - DEF * \$ * +$

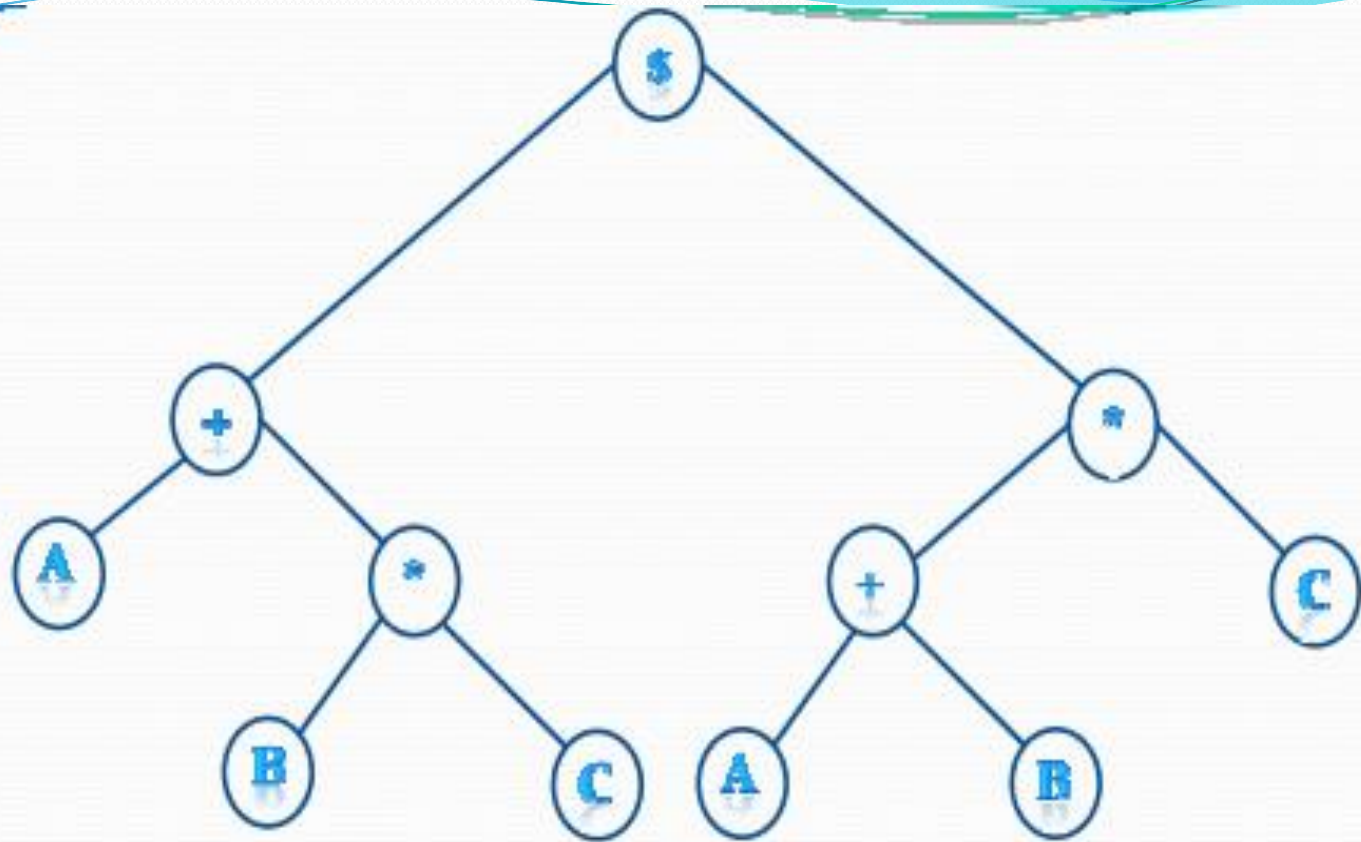
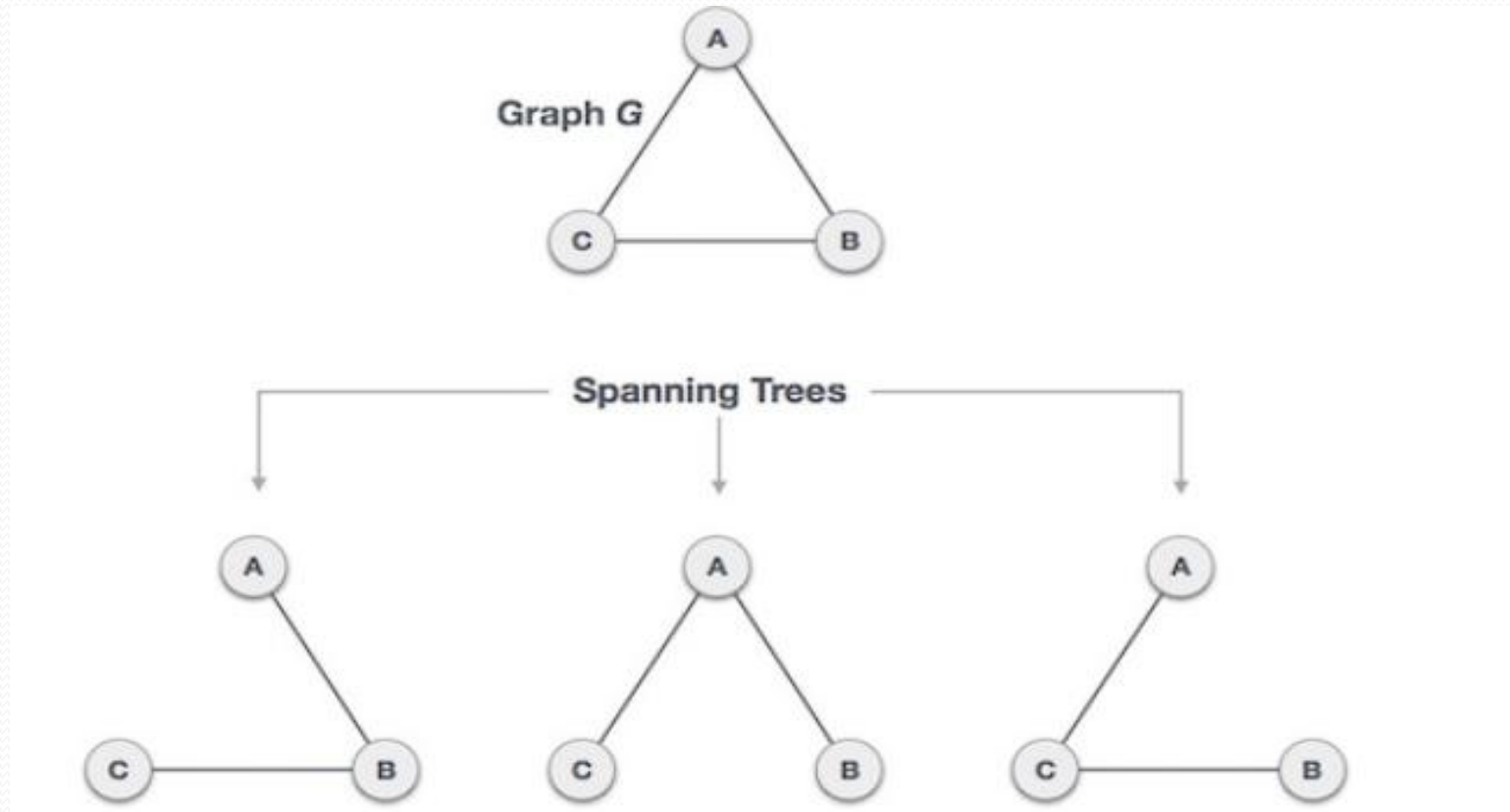


Fig (d): $(A+B^*C) \$((A+B)^*C)$

Postfix: $ABC^*+AB+C^*\$$

Spanning Tree

- A spanning tree of a graph G is a tree which contains all the vertices of a graph.

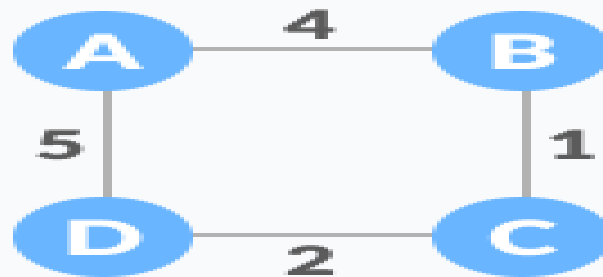


General Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G , have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Minimum Spanning Tree (MST)

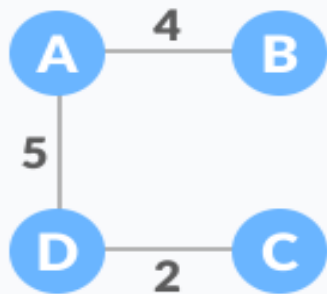
- A spanning tree with its total weight minimum is called minimum spanning tree.
- Given a weighted graph G , from which find several spanning tree and for MST, choose only that spanning tree whose total weight is minimum.



Weighted graph

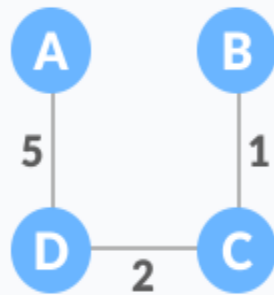
Minimum Spanning Tree (MST)

- The possible spanning tree of given weighted graph are:



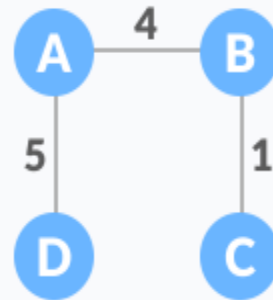
sum = 11

Minimum spanning tree - 1



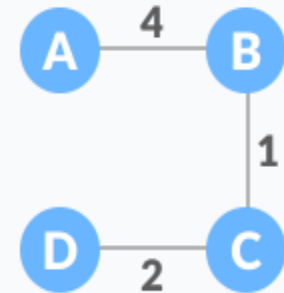
sum = 8

Minimum spanning tree - 2



sum = 10

Minimum spanning tree - 3



sum = 7

Minimum spanning tree - 4

The minimum spanning tree from the above spanning trees is:



sum = 7

Minimum spanning tree

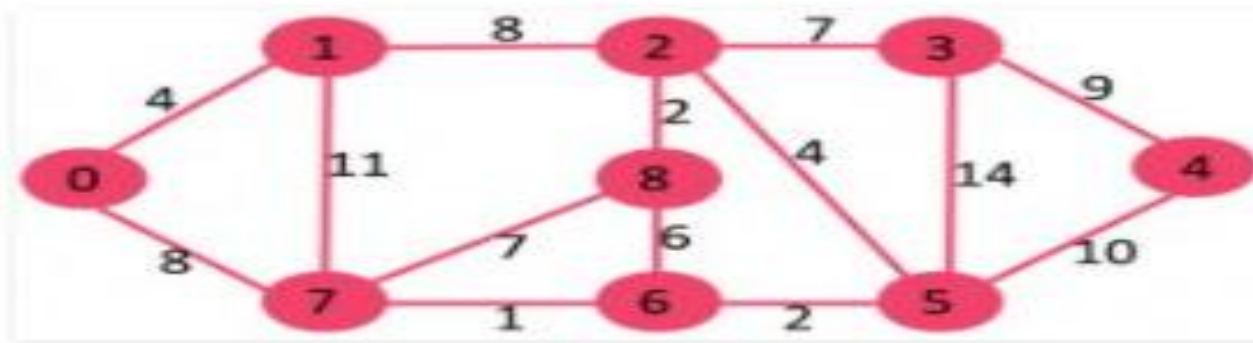
Kruskal's Algorithm

- Kruskal's Algorithm builds the minimum spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree.

Algorithm Steps:

1. *Sort all the edges in non-decreasing order of their weight.*
2. *Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If cycle is not formed, include this edge. Else, discard it.*
3. *Repeat step#2 until there are $(V-1)$ edges in the spanning tree.*

Example: Find MST using Kruskal's algorithm



Edges and their weights:

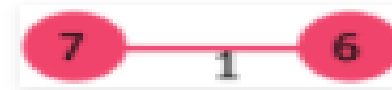
edge	0,1	1,2	2,3	3,4	4,5	5,6	6,7	7,0	1,7	2,5	2,8	3,5	6,8	7,8
Wt	4	8	7	9	10	2	1	8	11	4	2	14	6	7

Edges with increasing order of their weights:

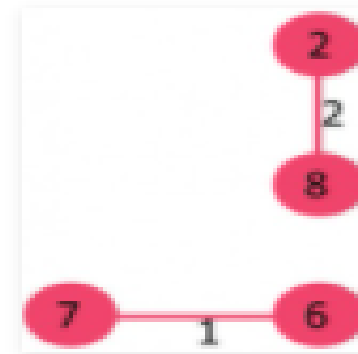
edge	6,7	2,8	5,6	0,1	2,5	6,8	2,3	7,8	7,0	1,2	3,4	4,5	1,7	3,5
Wt	1	2	2	4	4	6	7	7	8	8	9	10	11	14

Now pick all edges one by one from sorted list of edges

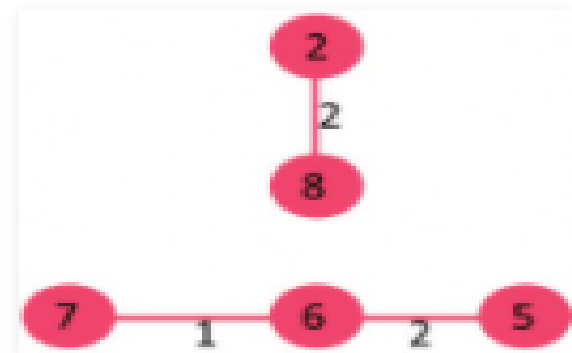
1. *Pick edge 7-6:* No cycle is formed, include it.



2. *Pick edge 8-2:* No cycle is formed, include it.

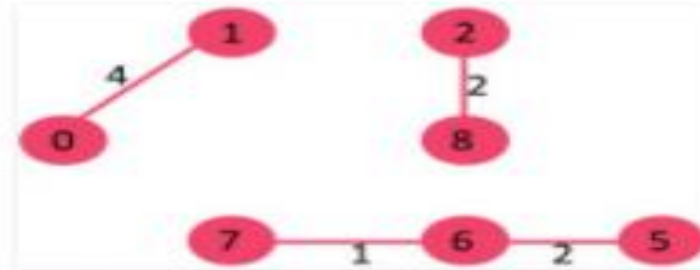


3. *Pick edge 6-5:* No cycle is formed, include it.



Example: Find MST using Kruskal's algorithm

4. Pick edge 0-1: No cycle is formed, include it.



5. Pick edge 2-5: No cycle is formed, include it.



6. Pick edge 8-6: Since including this edge results in cycle, discard it.

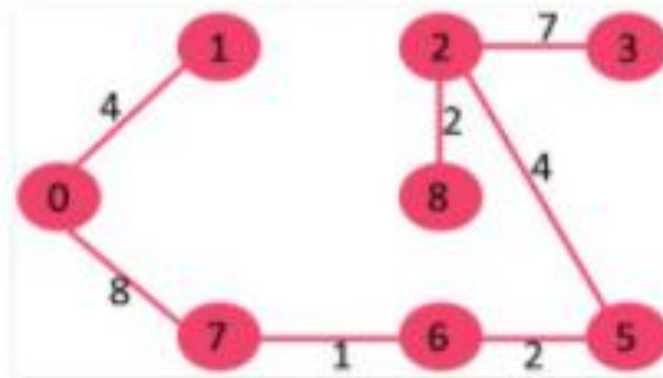
7. Pick edge 2-3: No cycle is formed, include it.



8. Pick edge 7-8: Since including this edge results in cycle, discard it.

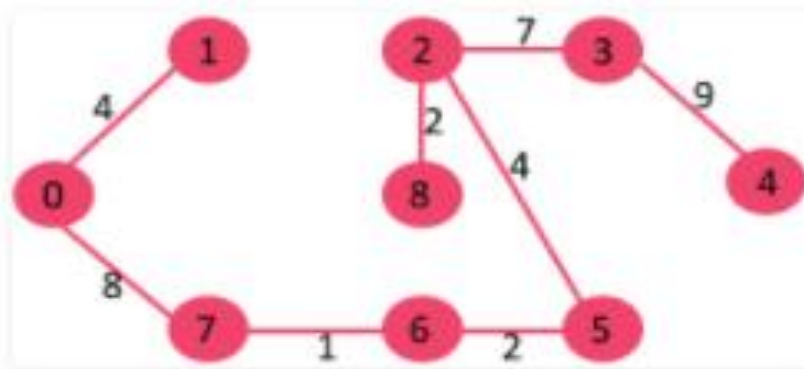
Example: Find MST using Kruskal's algorithm

9. Pick edge 0-7: No cycle is formed, include it.



10. Pick edge 1-2: Since including this edge results in cycle, discard it.

11. Pick edge 3-4: No cycle is formed, include it.



Since the number of edges included equals $(V - 1)$, the algorithm stops here.

- Total weight= $4+8+1+2+4+2+7+9=37$

Prim's algorithm

Step1: Select any vertex in G . Among all the edges which are incident to it, choose an edge of minimum weight and include it in the tree T .

Step 2: at each stage, choose an edge of smallest weight joining a vertex which is already in T and a vertex which is not included yet so that it does not form cycle. Then include it in T

Step 3: repeat until all the vertices of G are included with $(v-1)$ edges.



END